Balancing Coupling in Software Design

UO318869 – Ceyda Tolunay

UO277488 – Luis Sánchez de Posada Orihuela

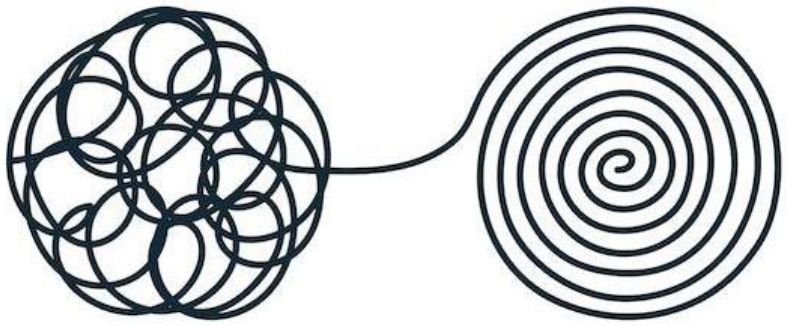
UO300737 – Ignacio Hoyos Diego

UO300084 – Sergio Seijo Martínez

UO299778 – Daniel Álvarez Menéndez

The Core Problem: System Complexity

- Coupling is not just a chapter in a book; it is the foundation of good software architecture.
- **The Goal:** We must manage and balance Coupling to keep complexity low.



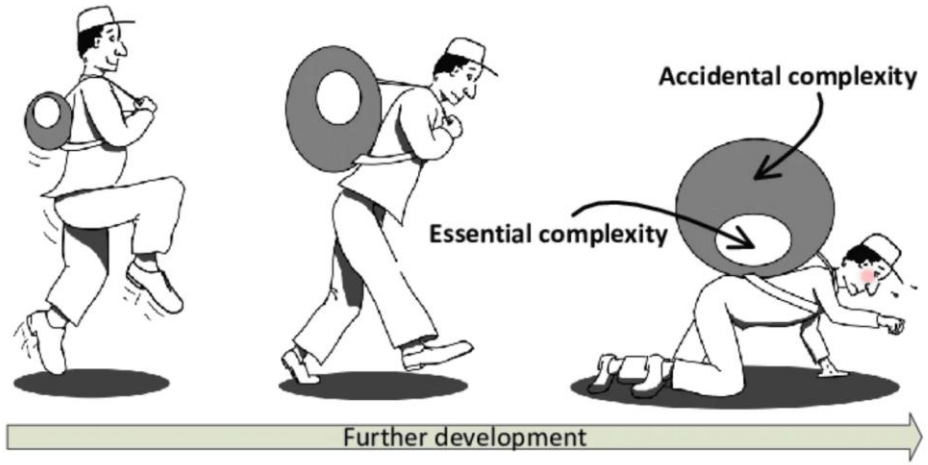
What is Complexity?

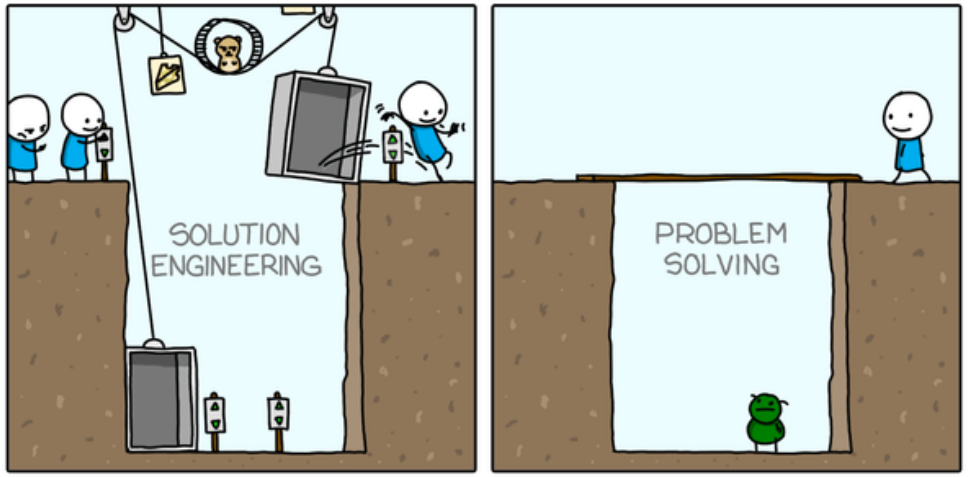
- It is the relationship between **action** and **result.**When a change is made in the system:
  - Predictable outcome → Not Complex
  - Unpredictable outcome → Complex
- The Nightmare Scenario: **"Big Ball of Mud"**

The Two Faces of Complexity

- **1. Essential Complexity**
  - Driven by the business domain
    - like Healthcare, Banking.
  - Unavoidable.
- **2. Accidental Complexity**
  - Driven by our bad design and architectural choices.
  - Avoidable!

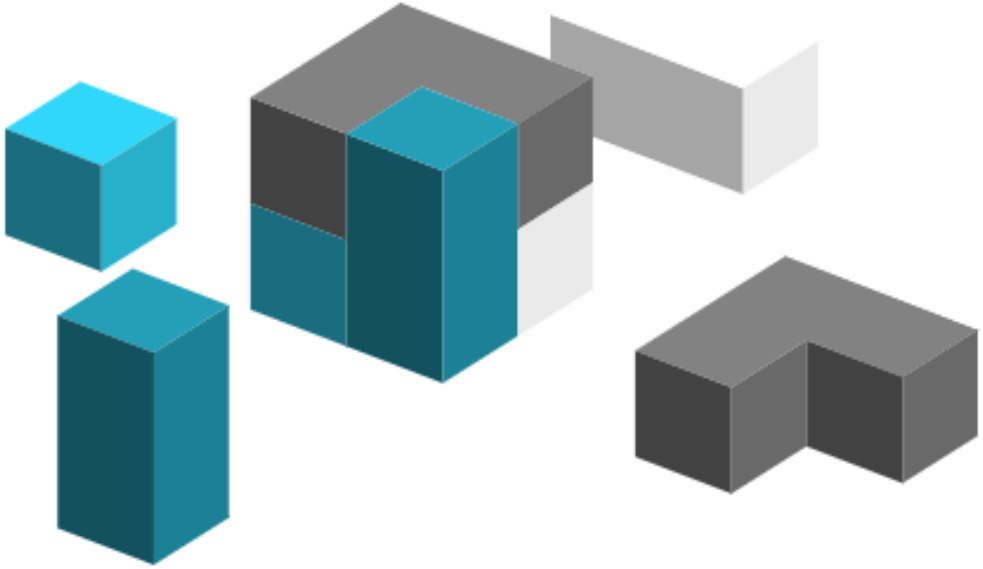
Our Goal: Minimize accidental complexity.





The Solution: Modularity

- Modularity is the exact opposite of complexity.
- It turns the **unpredictable** into **predictable**.
- Safely add new features without breaking existing ones.
- *The Next Step:* How do we define a "Module"?



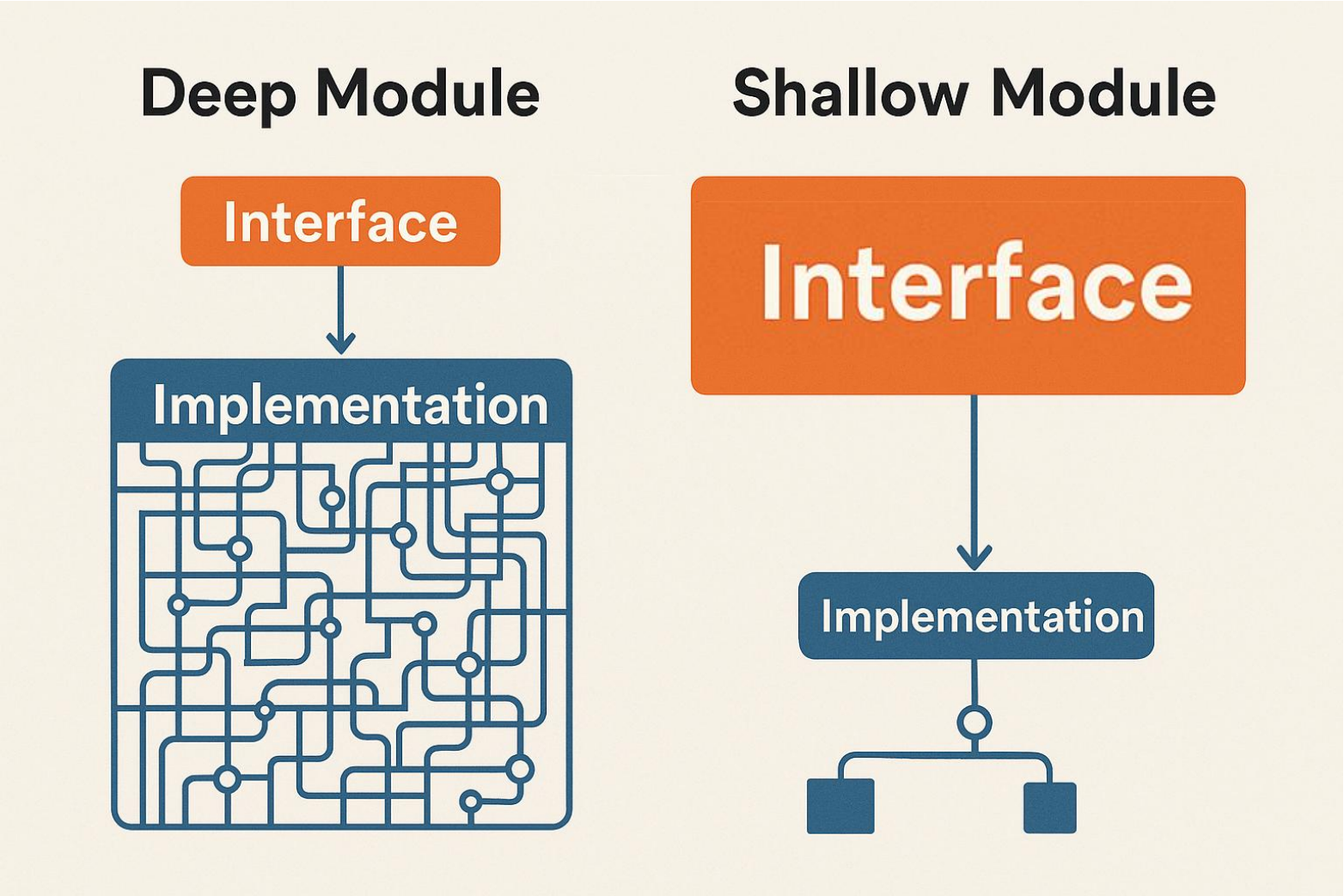
What is a module ?

WELL-DEFINED FUNCTIONALITY

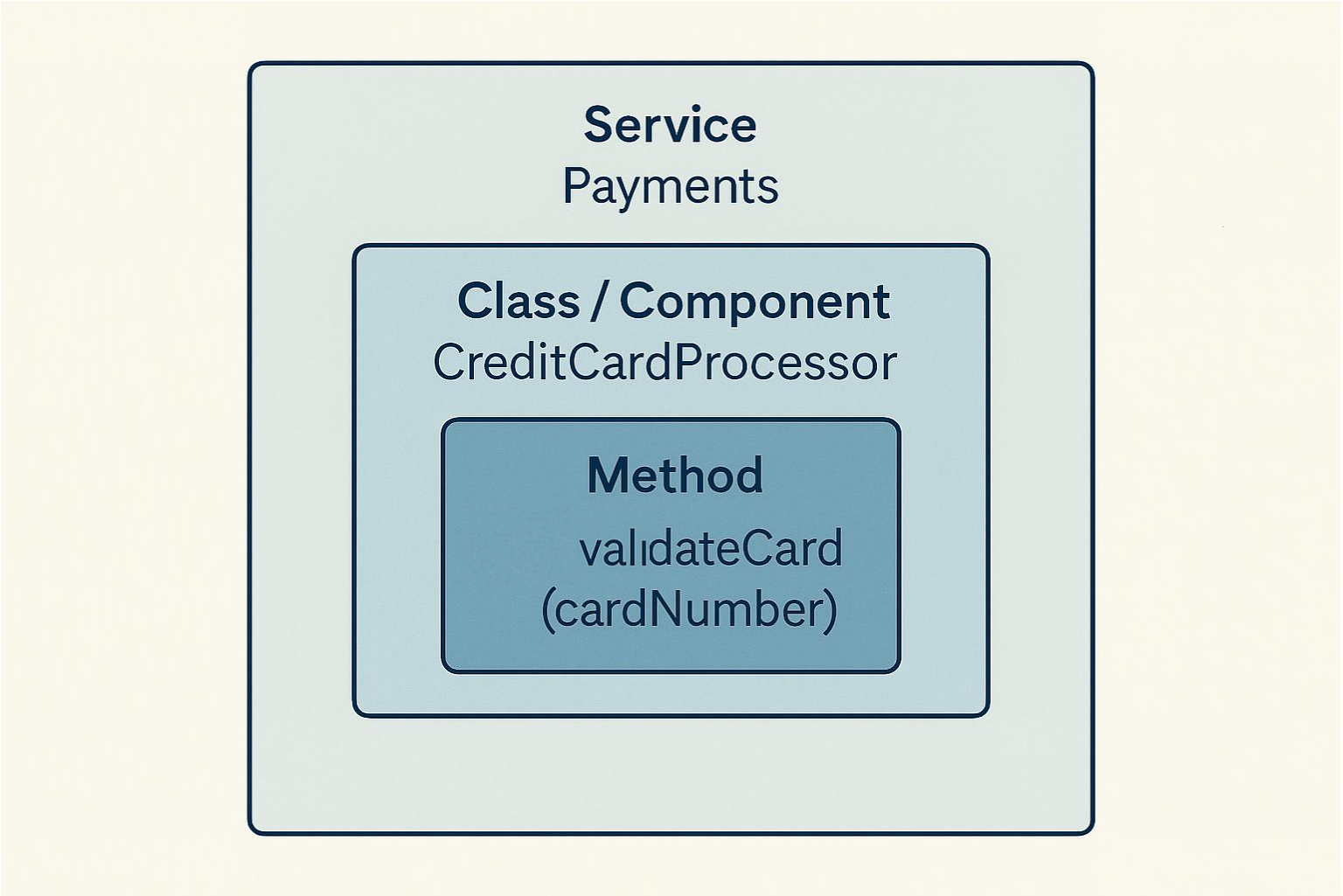
PUBLIC INTERFACE

INDEPENDENT COMPILE-ABILITY

Deep vs Shallow



The fractal nature of boundries



Drawing the line

Encapsulation Volatility

The “One Sentence Rule”

Logical vs Physical

What is coupling?

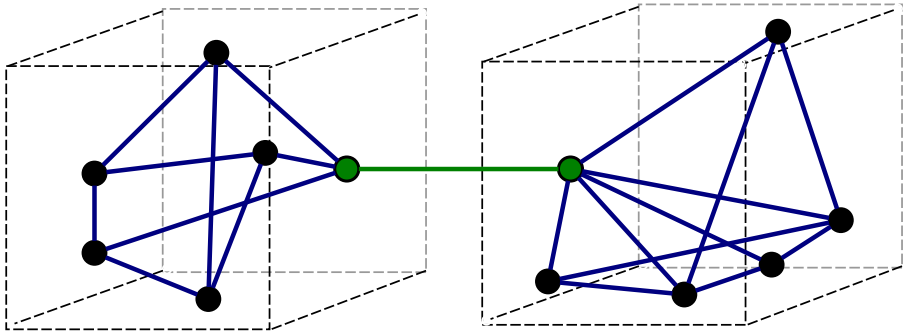
Degree of independece between modules/services/systems

Solutions

Remove all coupling

No system

Good design



Coupling = Knowledge Sharing

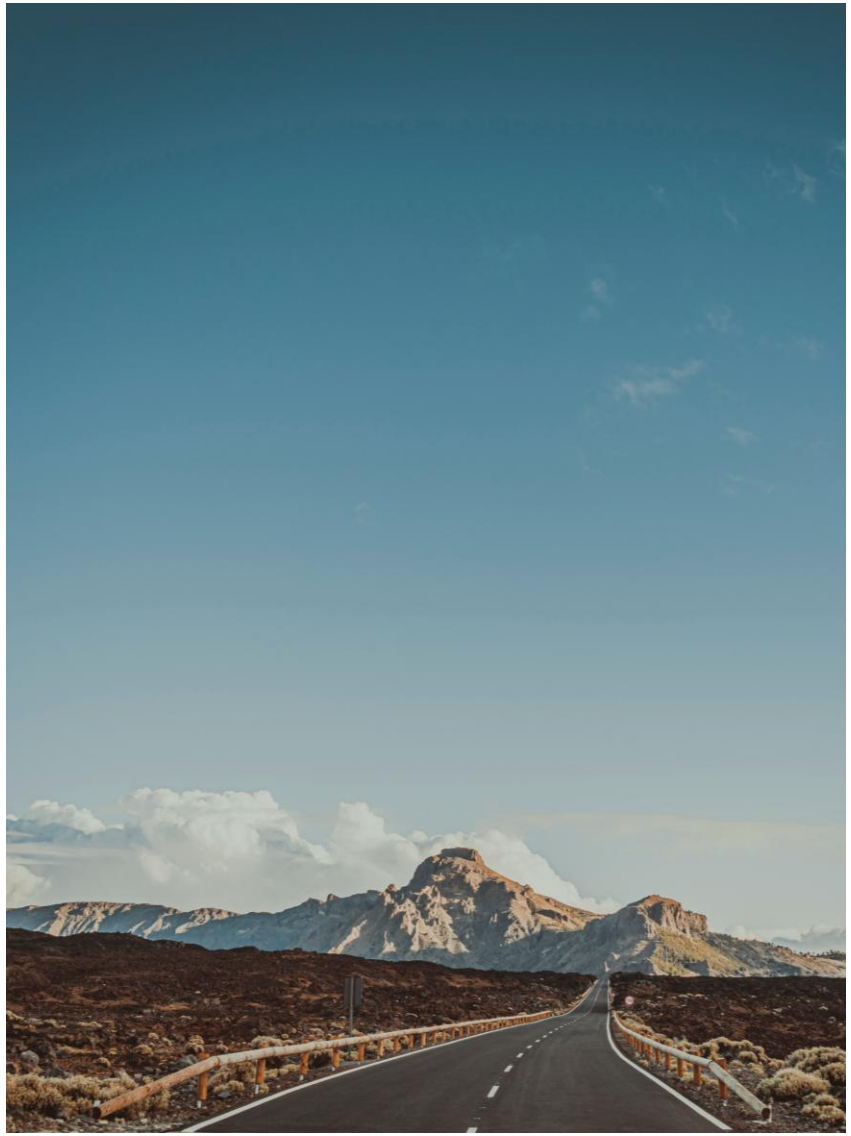
⚠ The more knowledge shared...

+ More cascading changes

+ More difficult to upgrade/scalate

Integration Strength

1	Contract Coupling	Low risk
2	Model Coupling	Moderate
3	Functional Coupling	High
4	Intrusive Coupling	Critical



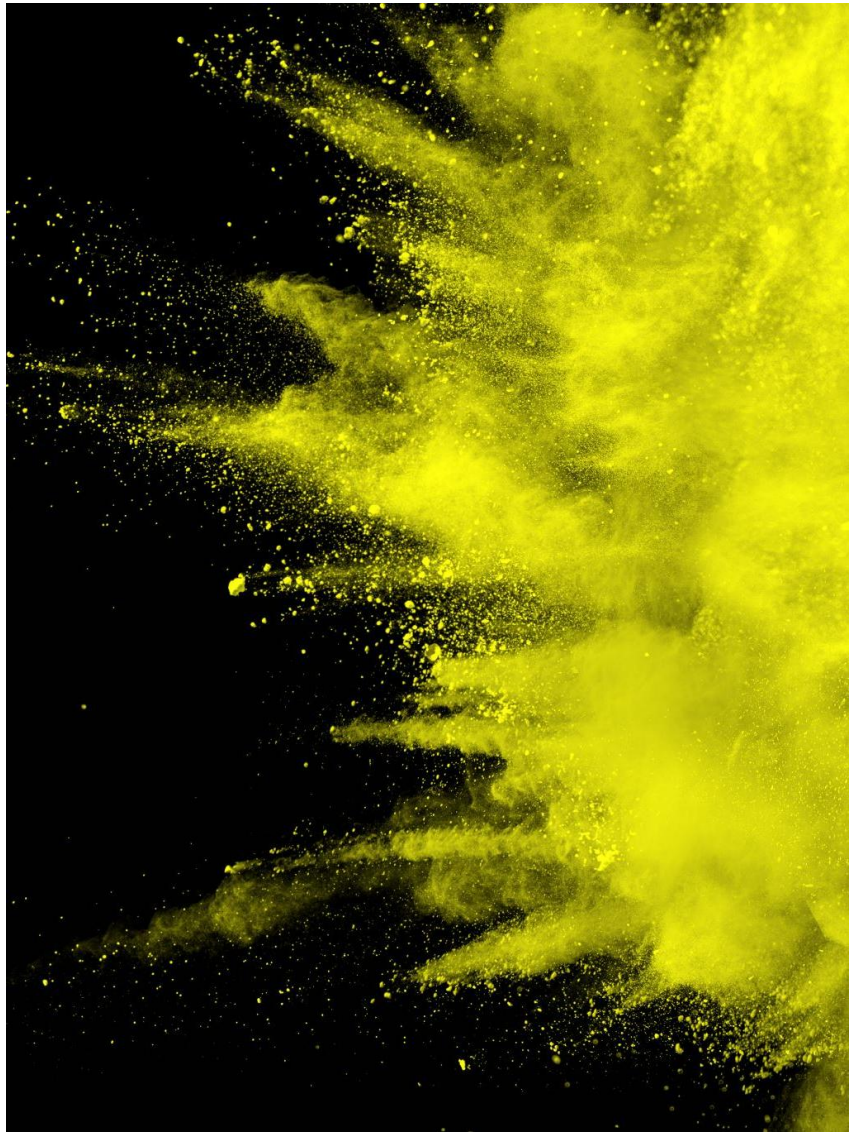
Distance

- **Definition:** The operational or organizational proximity between coupled modules.
- **The Scale of Distance:**
  - *Intra-process:* Same module/team (Low distance).
  - *Inter-process:* Different services/teams (Medium distance).
  - *External:* Third-party vendors/Global teams (High distance).
- **Key Insight (from Vlad):** Higher distance = Higher coordination effort.
- **The Risk:** When distance is high, you might not even know a breaking change is coming.

Volatility – The Rate of Change

- **Definition**: How often the upstream module (the one you depend on) changes its code or contract.
- **The Stability Paradox**: Coupling is NOT always a problem if the target is stable (e.g., standard libraries).
- **Measuring Volatility**:
  - Use **DDD Subdomains** to evaluate the rate of change. (Domain Driven Design).
  - **Core Domains = High Volatility** (Frequent changes).
  - **Generic Subdomains = Low Volatility** (Stable).
- **The Formula of Pain**:

  High Volatility + High Strength = Constant cascading changes

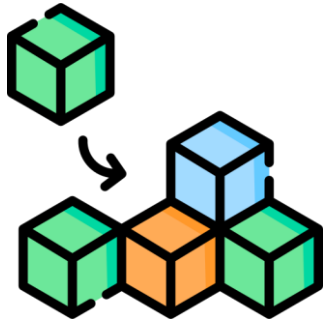


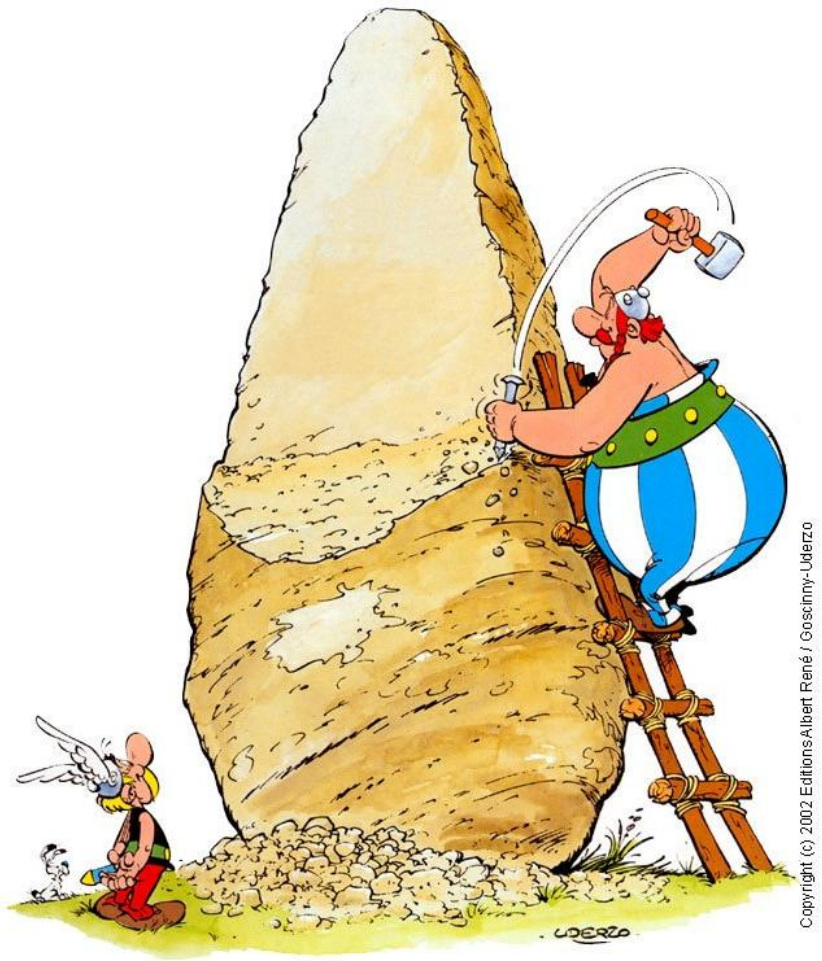
Monolithic Architecture Strategies

- **Accept Higher Integration Strength**

- **Embracing High Cohesion**

- **Fractal Boundaries**

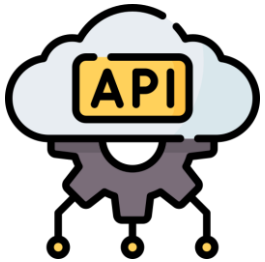




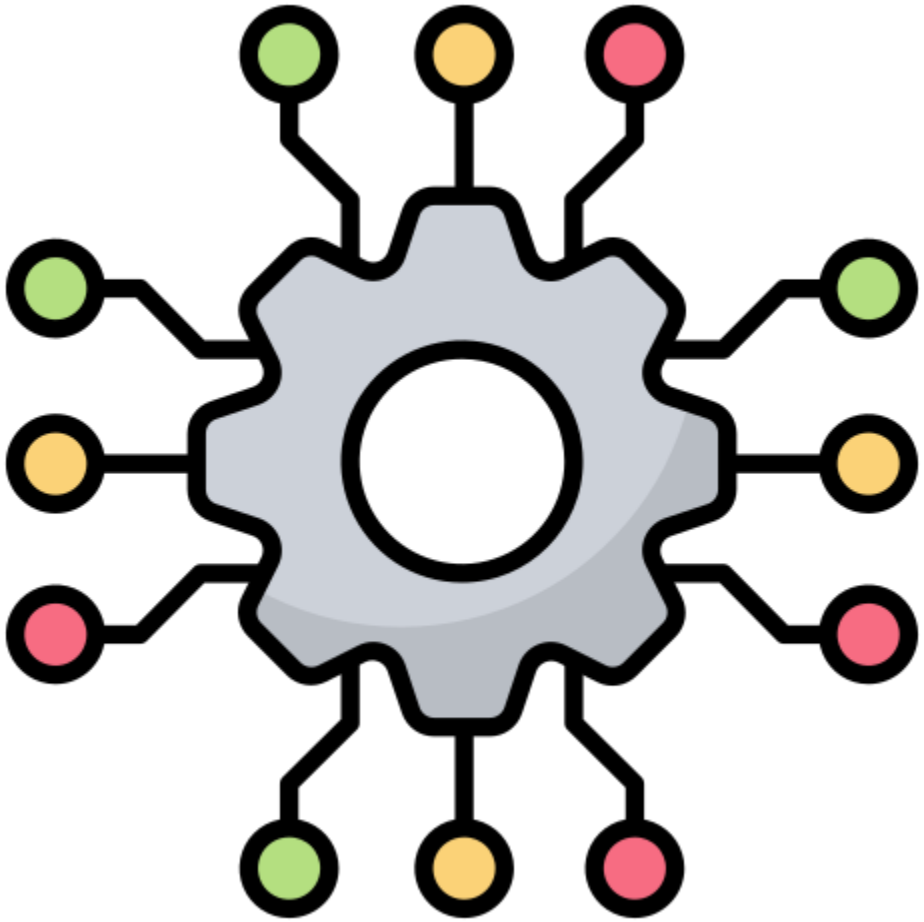
MicroServices Architecture Strategies

- Minimizing Integration Strength

- Public Interface Enforcement



- Avoiding Cascading Changes



Real-World Balancing (The "Heuristic")

---------- Knowledge VS Distance----------

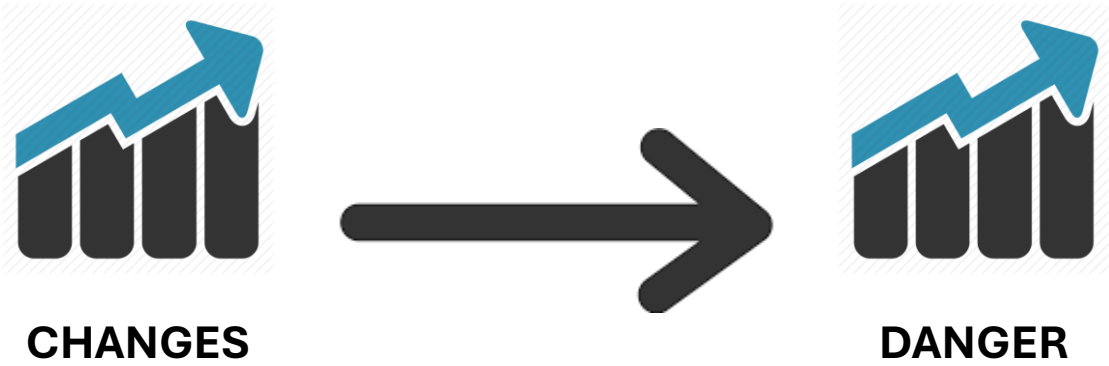
Knowledge is High

DECREASE DISTANCE

Distance is High

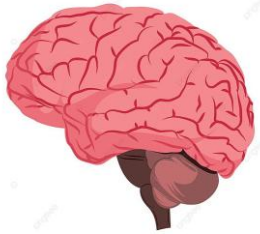
DECREASE KNOWLEDGE

Consider Volatility



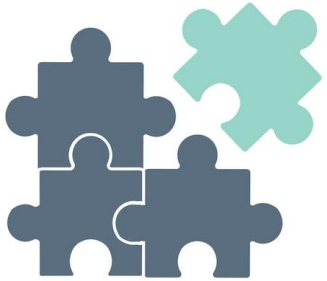
To Sum Up...

- **From Complexity to Predictability**



- **Modularity as the Goal**

- **Fractal Nature of Design**



Thank You For Listening!

Any questions?